



Project Report

Option: Artificial Intelligence and Data Science

Knowledge-Based Reinforcement Learning for Playing Zork

Prepared by:

TAHIR Abderrahmane
El Mehdi
SAIDANI Yasser
ANANI Moncef
TADJ Mohamed Walid

Supervised by:

BOUSMAHA Rabab
ELHANANI Souad

Academic Year: 2024/2025

Acknowledgements

First and foremost, we would like to thank Allah, the Almighty, for granting us the strength, courage, and patience to carry this work through to completion.

We would like to express our sincere gratitude to our supervisors, Ms. BOUSMAHA Rabab and Ms. ELHANANI Souad, for their competent guidance, invaluable support, and continuous encouragement throughout this project. Their attentiveness, scientific rigor, and insightful feedback have been essential in refining our work and improving its overall quality.

We would also like to extend our heartfelt thanks to all the faculty members and staff of the École Nationale Supérieure en Informatique of Sidi Bel Abbès, for their unwavering support, valuable advice, and commitment to delivering a multidisciplinary education of the highest quality.

Finally, we wish to thank everyone who contributed, directly or indirectly, to the completion of this project. Your help, dedication, and support have been deeply appreciated.

Abstract

Recent years have witnessed significant advances in reinforcement learning (RL) for natural language understanding, notably in LLMs that incorporate user feedback to refine their outputs. A related domain where language and feedback interplay is critical is text-based games, where agents must learn to navigate complex, partially observable environments through textual interactions. Games like *Zork* offer a rich testbed for combining RL and natural language processing (NLP), as they can be modeled as Partially Observable Markov Decision Processes (POMDPs) characterized by sparse rewards, ambiguous state representations, and long-term temporal dependencies. These challenges make text-based games an ongoing research subject in interactive AI. In this project, we propose an architecture that leverages structured knowledge via knowledge graphs (KGs) with the use of both NLP and RL to train an agent capable of achieving positive rewards in such environments. Our approach demonstrates promising results on *Zork*, reinforcing the value of integrating RL with structured and semantic language representations.

Keywords: Reinforcement Learning, Natural Language Processing, Interactive Fiction, Zork, Knowledge Graph.

Résumé

Récemment, des avancées majeures ont été réalisées dans l'apprentissage par renforcement (RL) appliqué à la compréhension du langage naturel, notamment avec les grands modèles de langage (LLM) intégrant le retour utilisateur pour affiner leurs réponses. Un domaine connexe où l'interaction entre langage et feedback est cruciale est celui des jeux textuels, où les agents doivent apprendre à naviguer dans des environnements complexes et partiellement observables via des interactions textuelles. Des jeux comme *Zork* constituent un terrain d'expérimentation riche pour combiner RL et traitement automatique du langage naturel (TALN), car ils peuvent être modélisés comme des processus de décision de Markov partiellement observables (POMDP) caractérisés par des récompenses rares, des états ambigus et de longues dépendances temporelles. Ces défis font des jeux textuels un sujet de recherche actif en IA interactive. Dans ce projet, nous proposons une architecture qui exploite la structuration des connaissances via des graphes de connaissances (KG) en combinant TALN et RL pour entraîner un agent capable d'obtenir des récompenses positives dans ces environnements. Notre approche montre des résultats prometteurs sur *Zork*, soulignant l'intérêt d'intégrer RL avec des représentations linguistiques structurées et sémantiques.

Mots-clés : Apprentissage par renforcement, Traitement automatique du langage naturel, Fiction interactive, Zork, Graphe de connaissances.

Contents

1	General Introduction	7
1.1	Partial Observability	7
1.2	Reward Sparsity	7
1.3	Long-term Dependencies	8
1.4	Natural Language Understanding	8
2	State of the Art	9
3	Architecture Detail	10
3.1	Input Representation	10
3.1.1	T5-OBSRVR	10
3.1.2	Graph Construction and Update	11
3.1.3	Relational Graph Convolutional Network (R-GCN)	11
3.1.4	Attention Pooling	12
3.2	Action Selection	12
3.2.1	Action Scorer	13
3.2.2	Critic	14
3.3	Policy Update	14
3.3.1	GAE Calculation	14
3.3.2	Advantage Normalization	15
3.3.3	Value Loss	15
3.3.4	Policy Loss	15
3.3.5	Final Loss	15
3.4	Architecture Choices	15
3.4.1	Partial Observability	15
3.4.2	Reward Sparsity	16
3.4.3	Long Time Dependencies	16
3.4.4	Natural Language Understanding	16
4	Implementation Detail	17
4.1	T5-OBSRVR	17
4.1.1	Dataset	17
4.1.2	Fine Tuning	17
4.1.3	Algorithm	18
4.1.4	Caching	18
4.2	Grapher	18
4.2.1	DGL	18
4.2.2	Update Algorithm	19
4.3	TextEnv	19
4.4	Mapping	19
4.4.1	Node Mapping	19
4.4.2	Relation Mapping	19
4.4.3	Action Mapping	19
4.5	Saving	20
4.6	Logging	20
4.7	Training	20

5	Experimentations and Results	21
5.1	Experimental Setup	21
5.2	Hyperparameter Tuning and Training Detail	21
5.3	Results	21
6	Conclusion	25
	References	26

1 General Introduction

Text-based games (also known as Interactive Fiction) are a form of games that were popular in the late 70s and mid 80s where interactions between the player and the game environment are entirely based on text input/output where the game describes the current state of the game (location, surroundings, events, ...) to the player and he responds with the appropriate action and the interactions go on until a final state is reached either by winning or losing.

Interactive fiction has been the subject of research for many years and it's still ongoing due the challenges it poses for learning agents which make it a rich testbed for Natural Language Understanding, Long-term memory and sequential decision-making.

The main challenges are outlined below:

1.1 Partial Observability

Text-based games can be modeled as Partially Observable Markov Decision Processes (POMDPs) which means that the returned observation by the game environment only represents a small part of the world state and not the entire view, unlike fully observable games like Tic-Tac-Toe where we can see how the grid changes with each move, in textual games, whenever we advance in the game world, the environment can only provide information about our current location, the things that are surrounding us and in our inventory and the impact made by the last move. This makes it hard for the rl agent to infer correct action based only on this information.

<i>Action:</i> CLIMB TREE
<i>Observation:</i> You are about 10 feet above the ground nestled among some large branches. There is a small nest here with a jeweled egg inside.
<i>Action:</i> TAKE EGG
<i>Observation:</i> You carefully remove the jeweled egg from the nest. It is surprisingly heavy.

Figure 1: Example of an interaction in Zork1

1.2 Reward Sparsity

Unlike typical games, Interactive Fiction require a sequence of correct actions in order to reach the goal in which only the last action is rewarded by the environment, this create an unbalance in action scoring where the last one only would seem to be gotten us the reward, and the same thing also applies for negative rewards where bad actions that result in dying can be ignored since only the last one caused death.

The absence of immediate action rewarding disturbs the agent's policy and causes bad decision making.

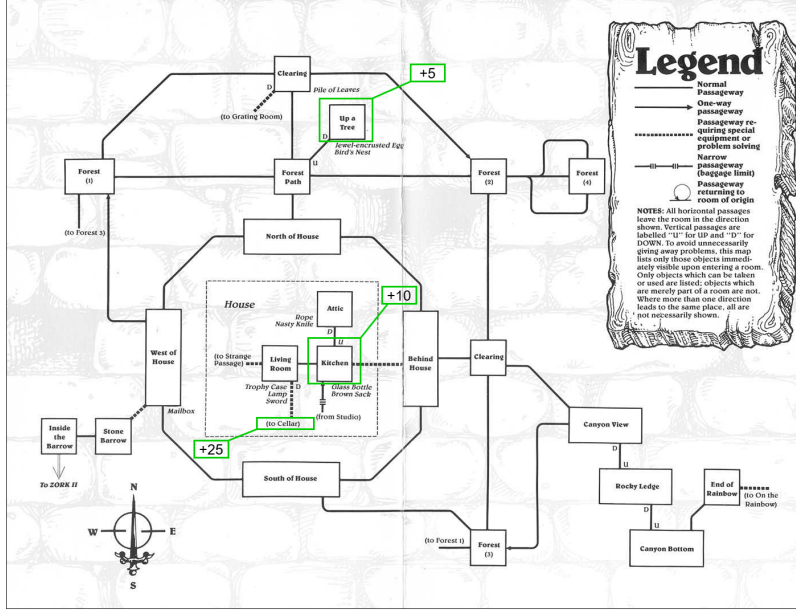


Figure 2: Rewards Scattering in Zork1

1.3 Long-term Dependencies

Another aspect to look out for is the long time dependencies between taken actions where they can be related even if a considerable time gap was present between them, this can include path selection, taken or missed items or even impact on non playable characters (NPCs).

1.4 Natural Language Understanding

The most important part which represents the initial step in our modeling is how to transform the environment output into a representation that our agent can leverage to understand the game state and use it for decision making. Since the only way to capture information about the game, this part is crucial and must be done with care.

With the presence of these challenges, choice of model design and architecture should be precise and must resolve the issues carried by them.

2 State of the Art

Upon literature review, we have seen numerous approaches that try to solve textual games and overcome challenges linked to them. For instance, Kartihik et al. (2015) proposed a deep reinforcement learning framework combining two modules: Representation Generator which used Long Short-Term Memory (LSTM) to convert textual descriptions into semantic vector representations and an Action Scorer which is deep neural network (DNN) that evaluate possible actions based on the state representation.

He et al. (2016) proposed a novel architecture Deep Reinforcement Relevance Network (DRRN) that uses separate neural networks to embed state and action texts into continuous vector spaces; this contrasts with traditional DQNs that concatenate state-action inputs or assume fixed action sets.

Direct description evaluation was not sufficient to address complex relationships in interactive fiction, so another approaches emerged to better represent the state; Ammanabrolu & Riedl, (2019) proposed KG-DQN where they used knowledge graphs (KGs) to enhance state representation by building a KG and updating it every step and combining a Graph Attention Network (GAT) for graph embedding with a bidirectional LSTM for textual description into a State Encoder, and an LSTM for action encoding and compute the action scores with dot product similarly to DRRN.

Ammanabrolu & Hausknecht (2020) introduced Advantageous Actor Critic to the previous approach by introducing an Action Decoder that uses a sequence of decoders to select the appropriate action template and select the best action combination with a critic that outputs a value for the given state.

Some notable attempts were done for solving text-based games with Large Language Models (LLMs) such as Wenlong Huang et al. (2022) that used Zero-Shot Planning with GPT-3 and have shown that LLMs are able to infer implicit game rules and commonsense knowledge without training. Shunyu Yao et al. (2022) proposed ReAct, an LLM framework that interleaves reasoning and acting and outperforms RL agents in some benchmarks.

Chhikara et al. (2023) introduced a new framework for improving AI agents playing interactive fiction by injecting domain knowledge in the form of Memory of Correct Action (MCA) and commonsense knowledge via object affordances.

Multiple frameworks were introduced for benchmarking and evaluating the performance of text-based games playing agents, notably; Côté et al. (2018) developed a framework TextWorld for procedurally generated textual games for AI training and testing. Hausknecht et al. (2020) introduced another framework JerichoWorld which is a unified interface for classic text-based games like Zork.

3 Architecture Detail

In this section, we detail our proposed architecture.

The general adopted idea is to using environment’s observation to build and update a knowledge graph that we embed and pass to the action decoder to select the most appropriate action as seen in Figure 3.

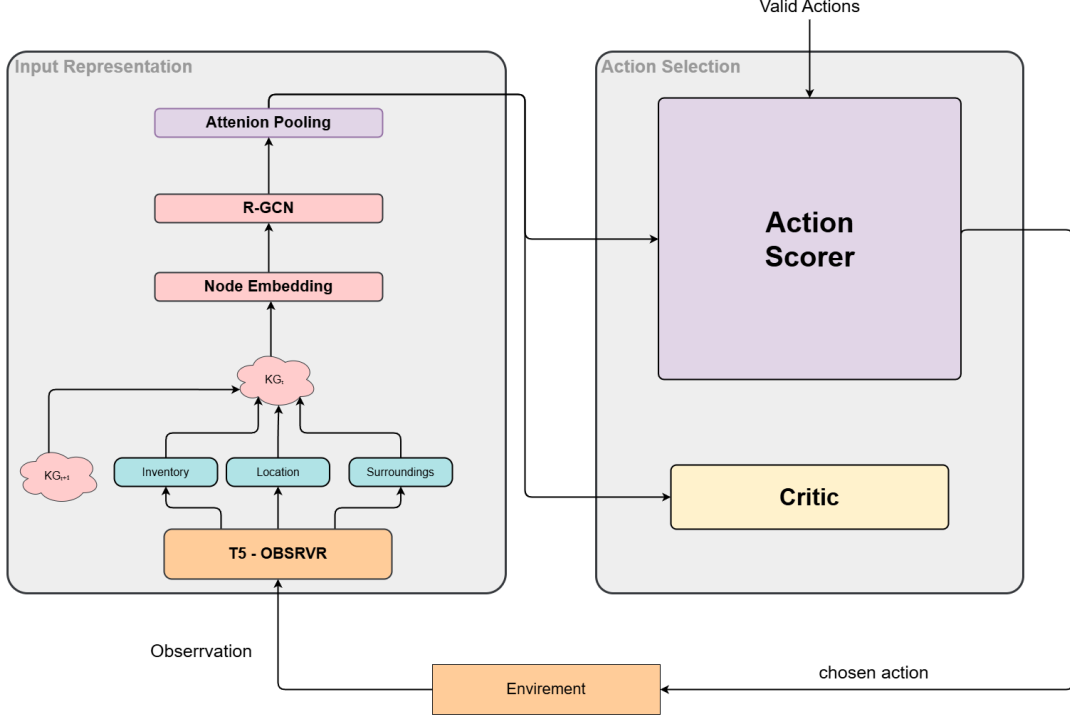


Figure 3: The Full Architecture

We describe each of the components below.

3.1 Input Representation

3.1.1 T5-OBSRVR

At each step, a full observation is received from the game environment that contains $O_t = (O_{osb}, O_{desc}, O_{inv}, O_{act})$ that is passed to the relation extraction module T5-OBSRVR which itself is composed of three T5 (Text-to-Text Transfer Transformer) models that outputs three different types of triplets:

- **Location Model:** takes the game’s textual description as input and returns the current location in the form of a triplet: $\langle \text{'you'}, \text{'in'}, \text{location} \rangle$.
- **Inventory Model:** takes the game’s inventory description as input and outputs the items in player’s inventory in the form of triplets: $\langle \text{'you'}, \text{'have'}, \text{item} \rangle$.
- **Surroundings Model:** from the game’s textual description, it returns the items surrounding the player (in the same location as him) in the form of triplets: $\langle \text{item}, \text{'in'}, \text{location} \rangle$.

We also use simple rules to extract the relation between past and current locations in the form of a triplet: $\langle \text{new_location}, \text{relation}, \text{past_location} \rangle$.

3.1.2 Graph Construction and Update

The knowledge graph state is kept by the module Grapher which tracks updates and new triplets in our KG eliminating unnecessary entries and keeping it coherent and ready for visualization. At each step, Grapher takes as input the result of the last module and updates the KG accordingly.

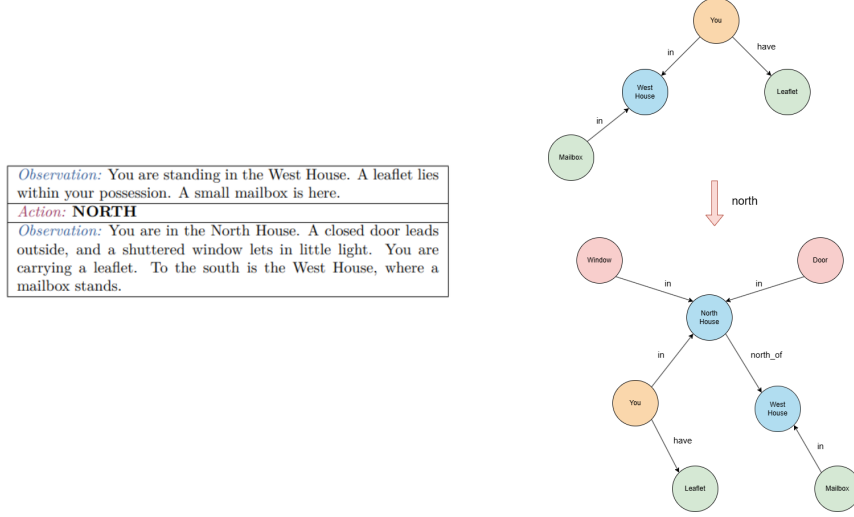


Figure 4: KG update example

3.1.3 Relational Graph Convolutional Network (R-GCN)

After having the current state's knowledge graph as follows:

$$G = \{(s_i, r_i, o_i)\}_{i=0}^t \quad (1)$$

with

$$\begin{cases} s_i, o_i \in V \\ r_i \in E \end{cases}$$

We need to transform this representation to the appropriate input for R-GCN which is a graph structure composed of three lists: sources, destinations and relations as follows:

$$G = \begin{cases} [s_1, s_2, s_3, \dots, s_t] \\ [o_1, o_2, o_3, \dots, o_t] \\ [r_1, r_2, r_3, \dots, r_t] \end{cases} \quad \text{where } i = 1, \dots, t \quad (2)$$

R-GCN also requires node features that we get by passing the node indices by an Embedding Layer, and relation types indices.

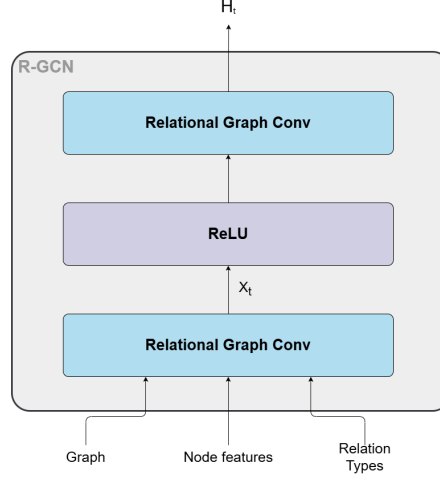


Figure 5: R-GCN Module Architecture

These informations are passed then to R-GCN which processes the relation between every node and its neighbouring nodes, in addition to its proper carried information according to the following expression:

$$h_v^{(l+1)} = \sigma \left(\sum_{r \in \mathcal{R}} \sum_{u \in \mathcal{N}_r(v)} \frac{1}{c_{v,r}} W_r^{(l)} h_u^{(l)} + W_0^{(l)} h_v^{(l)} \right) \quad (3)$$

And the final output of R-GCN will be:

$$H_t = [h_{v_1}^{(l')}, h_{v_2}^{(l')}, \dots, h_{v_n}^{(l')}] \in \mathbb{R}^{|V_t| \times d} \quad (4)$$

With l' being the final layer in R-GCN, $|V_t|$ the number of nodes and d the embedding dimension.

3.1.4 Attention Pooling

After getting the final graph embedding, we construct a vector that we can pass over to the action decoder; we use attention pooling to give more importance to some nodes (eg. the node ⟨you⟩ should have more weight than node ⟨tree⟩).

$$Z_t = \sum_v \alpha_t[v] H_t[v] \in \mathbb{R}^d \quad (5)$$

With α_t being a learnable weight for each node v .

3.2 Action Selection

This module represents the core of RL in the architecture and is composed of an Action Scorer and a critic.

3.2.1 Action Scorer

Main RL component, it is responsible for generating a probability distribution over the action space, it follows an Encoder-Decoder architecture composed of:

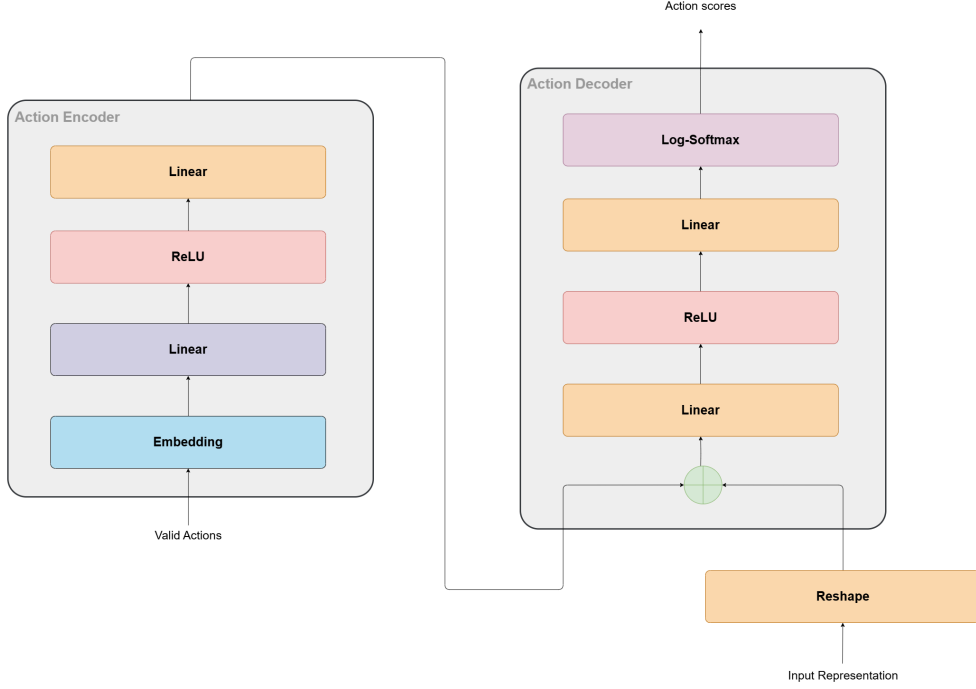


Figure 6: Action Scorer Full Architecture

Action Encoder The Encoder part of the scorer, it takes as input the valid actions in this state, embeds them and passes them through a projection network to get the valid actions embedding.

$$\begin{aligned}
 &\text{Having } A_t = \{a_1, a_2, \dots, a_N\} \text{ as valid actions,} \\
 &E_0 = \text{Embedding}(A_t) \in \mathbb{R}^{|A_t| \times d'}, \\
 &E_1 = \text{Linear}(E_0) \in \mathbb{R}^{|A_t| \times d}, \\
 &E_2 = \text{Linear}(\text{ReLU}(E_1)) \in \mathbb{R}^{|A_t| \times d}.
 \end{aligned} \tag{6}$$

Action Decoder The Decoder part of the scorer, it takes as input both the embedding of valid actions that we get from the encoder and the input representation z_t and outputs a probability distribution over the valid actions.

$$\begin{aligned}
 &Z'_t = \text{Reshape}(Z_t) \in \mathbb{R}^{|A_t| \times d}, \\
 &D_0 = Z'_t \oplus E_2 \in \mathbb{R}^{|A_t| \times 2d}, \\
 &D_1 = \text{Linear}(D_0) \in \mathbb{R}^{|A_t| \times 256}, \\
 &D_2 = \text{Linear}(\text{ReLU}(D_1)) \in \mathbb{R}^{|A_t|}, \\
 &P_t = \text{Log-softmax}(D_2) \in \mathbb{R}^{|A_t|}.
 \end{aligned} \tag{7}$$

3.2.2 Critic

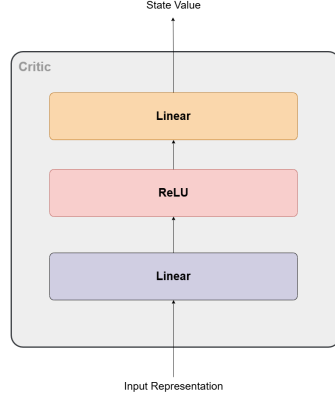


Figure 7: Critic Architecture

This module represents a key part in the RL policy, it is responsible for returning the value of the state which is mandatory in the learning process of the agent.

$$\begin{aligned} V_1 &= \text{Linear}(Z_t) \in \mathbb{R}^{\frac{d}{2}}, \\ V_2 &= \text{Linear}(\text{ReLU}(V_1)) \in \mathbb{R}. \end{aligned} \quad (8)$$

3.3 Policy Update

In this section we establish the mathematical detail of the reinforcement learning policy.

As input, we have the following:

$$\begin{aligned} R &= \{r_1, r_2, \dots, r_T\}, \\ V &= \{v_1, v_2, \dots, v_T\}, \\ D &= \{d_1, d_2, \dots, d_T\}. \end{aligned} \quad (9)$$

With R being the rewards we get each step by the environment, V being the values returned by the critic for each state, D being the 'done' flag for each step which is set to 1 once the episode finishes, otherwise 0; and T is the number of steps at this episode.

We use Generalized Advantages Estimation (GAE) in our policy to estimate the advantage of each step which we use to calculate the loss and update the model's parameters.

3.3.1 GAE Calculation

We iterate through the steps and calculate the Temporal Difference (TD) error:

$$\delta_t = r_t + \gamma(1 - d_t)v_{t+1} - v_t \quad (10)$$

with γ being the discount factor.

We use the TD error then to estimate the advantage for that step:

$$\hat{A}_t = \delta_t + \gamma\lambda(1 - d_t)\hat{A}_{t+1} \quad (11)$$

where λ is a regularization constant controlling how much weight is given to longer vs shorter returns which gives a trade-off between variance and bias.

3.3.2 Advantage Normalization

After computing the entire episode advantages, we normalize them to stabilize training and keep it more consistent:

$$\hat{A}_t = \frac{\hat{A}_t - \mu(\hat{A}_t)}{\sigma(\hat{A}_t) + \epsilon} \quad (12)$$

where ϵ is a small close to zero constant.

3.3.3 Value Loss

In order to update the value network (Critic), we need to calculate its loss. We start by computing the target value:

$$\hat{R}_t = \hat{A}_t + V_t \quad (13)$$

this tells us how much the value should be since V_t is the returned term by the critic and the advantage is how much better/worse is the state compared to that return.

Then, we have the value loss using mean squared error (MSE) as follows:

$$\mathcal{L}_{\text{Value}} = \frac{1}{T} \sum_{t=1}^T (V_t - \hat{R}_t)^2 \quad (14)$$

3.3.4 Policy Loss

We use the policy gradient loss as follows:

$$\mathcal{L}_{\text{Policy}} = \frac{1}{T} \sum_{t=1}^T \log \pi(a_t | s_t) \cdot \hat{A}_t \quad (15)$$

3.3.5 Final Loss

For the final loss that we use to update our parameters, we simply combine the value and policy losses as follows:

$$\begin{aligned} \mathcal{L} &= \mathcal{L}_{\text{Policy}} + \frac{1}{2} \mathcal{L}_{\text{Value}} \\ &= \frac{1}{2T} \sum_{t=1}^T \left[(V_t - \hat{R}_t)^2 - 2 \log \pi(a_t | s_t) \cdot \hat{A}_t \right] \end{aligned} \quad (16)$$

3.4 Architecture Choices

In this section we will detail how our choice of components in the architecture serve to solve the challenges faced in text-based games:

3.4.1 Partial Observability

The choice of utilizing knowledge graphs serves as great power to raise difficulties faced with incomplete information about the game state, since we build and update the KG every step, more information is gathered about the full state and less of a problem partial observability is.

3.4.2 Reward Sparsity

The game returns rewards for achieving milestones and not for each correct action we take, this is called extrinsic rewards (game only feedback); so we implement intrinsic rewards: rewards we define that are added to the extrinsic rewards in order to overcome the issue of reward sparsity.

The intrinsic reward is defined by the following:

$$r_{\text{intrinsic}} = 0.1 \times T_{\text{new}} + 0.3 \times L_{\text{new}} \quad (17)$$

with T_{new} being the number of new triplets in KG and L_{new} being a boolean set to 1 if the current location is a new one otherwise 0.

It is then added to total reward as follows:

$$r_{\text{total}} = r_{\text{extrinsic}} + r_{\text{intrinsic}} \times \text{intrinsic-decay} \quad (18)$$

where intrinsic decay is a decay factor calculated dynamically every episode:

$$\text{intrinsic-decay} = \max \left(0.1, 1 - \frac{\text{episode number}}{\text{number of episodes}} \right) \quad (19)$$

We use this decay factor to encourage exploration at the beginning of training but decrease it later to focus more on the actual game reward and not the rewards we defined.

By introducing intrinsic reward, we are able to overcome the reward sparsity issue.

3.4.3 Long Time Dependencies

With the generalized advantages estimation, it is possible to alleviate this issue since it helps with long-term credit assignment (giving more credit to actions that lead later to better outcomes) and it also reduces variance in the policy updates.

3.4.4 Natural Language Understanding

Since the first and only information we get about the game state is in textual form, we used language models (T5 transformers) which we fine tuned in order to extract the information we need in graph construction that represents the initial input to our model.

4 Implementation Detail

4.1 T5-OBSRVR

In order to build and update the knowledge graph, a mechanism was needed to understand the text observations and extract appropriate triplets. Related works used rule-based extraction, OpenIE or fine tuned ALBERT as question answering language models. Our approach was creating T5-OBSRVR, a module based on three fine tuned t5 transformers to generate the triplets needed to update the KG each step. As seen earlier the first model extracts the current player’s location, the second returns its surroundings and the last one extracts items in his inventory.

4.1.1 Dataset

In order to fine tune the three models on such task, we combined two datasets:

- **TextWorld KG Dataset:** a dataset containing previous triplets, previous action, observation and target commands (adding or removal of triplets) for games in TextWorld environment.
- **JerichoWorld Dataset:** a dataset containing walkthroughs from JerichoWorld environment with graph construction.

We processed these datasets and combined them into one dataset containing:

- Textual observation from environment
- Surrounding objects
- Current location
- Inventory items

We further processed this dataset for each task having only the text observation as input and the target triplets as output, for location extractor the output is the location itself and for inventory or surroundings extractor the output is the items separated by —.

4.1.2 Fine Tuning

We chose the small variant of T5 because our task is very specific and not general (only extraction of location or items for each model) and because this variant is small and lightweight.

4.1.3 Algorithm

Algorithm 1 OBSRVR

Require: Textual description from game environment, Previous location, Previous action

Ensure: New Triplets

```
1: location  $\leftarrow$  LocationExtractor(description)
2: location_triplet  $\leftarrow$  ("you", "in", location)
3: inventory  $\leftarrow$  InventoryExtractor(description)
4: inventory_triplets  $\leftarrow$  []
5: for item in inventory do
6:   inventory_triplets.ADD(("you", "have", item))
7: end for
8: surroundings  $\leftarrow$  SurroundingExtractor(description)
9: surrounding_triplets  $\leftarrow$  []
10: for item in surroundings do
11:   surrounding_triplets.ADD((item, "in", location))
12: end for
13: triplets  $\leftarrow$  JOIN(location_triplet, inventory_triplets, surrounding_triplets)
14: if location  $\neq$  previous_location then
15:   relation  $\leftarrow$  GetRelation(previous_action)
16:   triplets.ADD((location, relation, previous_location))
17: end if
18: return triplets
```

4.1.4 Caching

Another crucial part in T5-OBSRVR is triplet caching which is a feature implemented by storing textual descriptions as keys and triplets as values; and for each step, we check if the input matches one of the descriptions, if yes return the triplets else pass by the models. This technique significantly reduces model running time by minimizing inference from t5 models which can take some time and especially since it is done for every step.

4.2 Grapher

The module used to keep track of the knowledge graph, it is also responsible for getting information from the graph that we need in processing while training.

4.2.1 DGL

The library we used to pass from triplets representation to something we use in the model is Deep Graph Library (DGL) which we integrated with the Grapher module to build the graph that we pass to the R-GCN.

4.2.2 Update Algorithm

Algorithm 2 Grapher Update

Require: New observed triplets

Ensure: Updated knowledge graph

```
1: triplets = []
2: for triplet in old_triplets do
3:   head, relation, tail  $\leftarrow$  triplet
4:   if head  $\neq$  "you" then
5:     triplets.ADD(triplet)
6:   end if
7: end for
8: for triplet in new_triplets do
9:   if triplet not in triplets then
10:    triplets.ADD(triplet)
11:   end if
12: end for
```

4.3 TextEnv

This module represents the interface to the environment, we initialize it via Jericho which itself is an interface for man crafted games such as Zork1.

We modify the stepping method and make it step by the given action, in addition to executing both the "look" and "inventory" command to return the full observation that we use in our RL model.

4.4 Mapping

Since we use embeddings in multiple stages (node embedding and action encoding), we ought to use three mappings that we keep track of and update throughout training:

4.4.1 Node Mapping

This is a game specific mapping, meaning each game should have its own mapping file containing the node itself as a string and its corresponding index with the exception of the node "you" which is constant throughout all ROMs.

4.4.2 Relation Mapping

Since we are using R-GCN which uses a fixed number of relations, it should be the same for our relation mapping. We define a fixed set of relations that we find in the triplets and store them in a common file called rel2id to use for all ROMs.

4.4.3 Action Mapping

The action mapping is used in the action encoder where we index the valid actions before embedding them and pass them to the projection layer.

4.5 Saving

In the training process we constantly save the weights in addition to the previously mentioned mappings via HuggingFace, each run of the model is saved in a separate repository to avoid any conflicts. This makes it easier to resume training whenever an issue occurs, especially because training takes up to days depending on the number of episodes.

4.6 Logging

Another key element is the logs, we save all runs on the same model run repository (each 100 episodes in a json file). We save for each episode the number of that episode, the final score and the detailed information for each step (observation, location, valid actions, the probability distribution over these actions, the chosen action and the reward yielded by the environment). This played a key role in monitoring the model and inspecting non wanted behaviour throughout training.

4.7 Training

For training, we define a Trainer class where we implement the RL loop with all modules connected, we also define the training arguments in a separate yaml file, this is done to make tuning easier and more modular.

5 Experimentations and Results

5.1 Experimental Setup

We trained our agent in the interactive fiction game Zork1 as the main benchmark due to its difficulty and complex reward system. The architecture detailed in this report has passed through different stages and has seen significant changes throughout the time of project with some modifications being more important than others.

Our main concern was getting a good score that the agent learned over training time and the behaviour it grew, since we wanted the model to understand the world dynamics, complex relationships and game rules.

5.2 Hyperparameter Tuning and Training Detail

For more than a month, we trained the model with different combinations of architectural details and hyperparameters. For instance, we tested the model on a simple policy gradient on immediate rewards from the environment, then introduced intrinsic motivation and finally settled on generalized advantage estimation with intrinsic rewards. We tried adding commonsense knowledge extracted from ConceptNet dataset as a form of knowledge injection seen in multiple papers but it added more noise to the KG and did not add much value to the agent. Adding the buffer replay was also something we tested but it did not appear to add anything to the agent.

Another crucial part was the action scorer which we changed a lot during trials from using frozen bert for action encoding with projection to a simpler embedding layer with projection because the actions are small and do not need a transformer to process them, the action decoder also changed from a simple cosine similarity to a learned similarity via a simple MLP. The critic also evolved from a single linear layer to a 2 layer mlp with activation.

When it comes to hyperparameter tuning, the task was easier due to the presence of a config file that we changed a bit for each run and monitored the changes carefully to understand what served the model the best. For instance, keeping the same dimension for the model from node embedding to scoring (eg. 256) gave better results than different dimensions. And for the learning rate, small values ranging from $1e-5$ to $5e-4$ gave better results.

5.3 Results

In our journey to reach the final results, we underwent extensive testing and iterative improvements. The earliest prototype models had a completely different architecture featuring learned representation via information bottleneck, frozen BERT for action embedding, and action scoring with cosine similarity. However, these models showed no significant signs of learning, exhibiting large instabilities and high variance in performance.

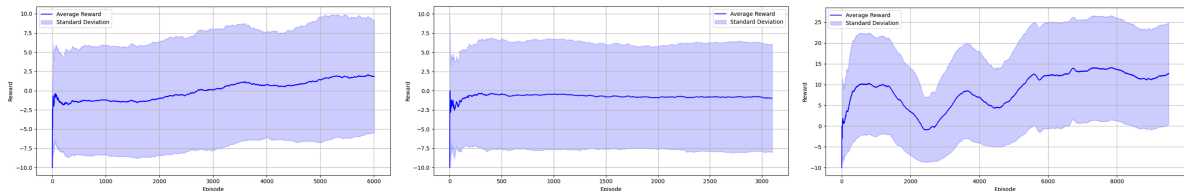


Figure 8: Early prototype models showing instability and high variance

After removing the representation learning component while keeping the attention mechanism, the model still converged to minimal rewards and the high variance persisted, indicating that the core architectural issues remained unresolved.

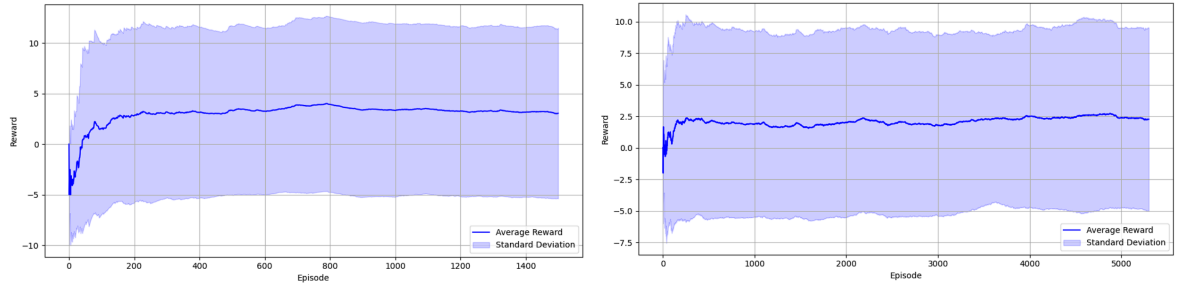


Figure 9: Results after removing representation learning

The removal of freezing in BERT did not provide any improvement to the model performance and maintained the same fundamental issues with convergence and variance.

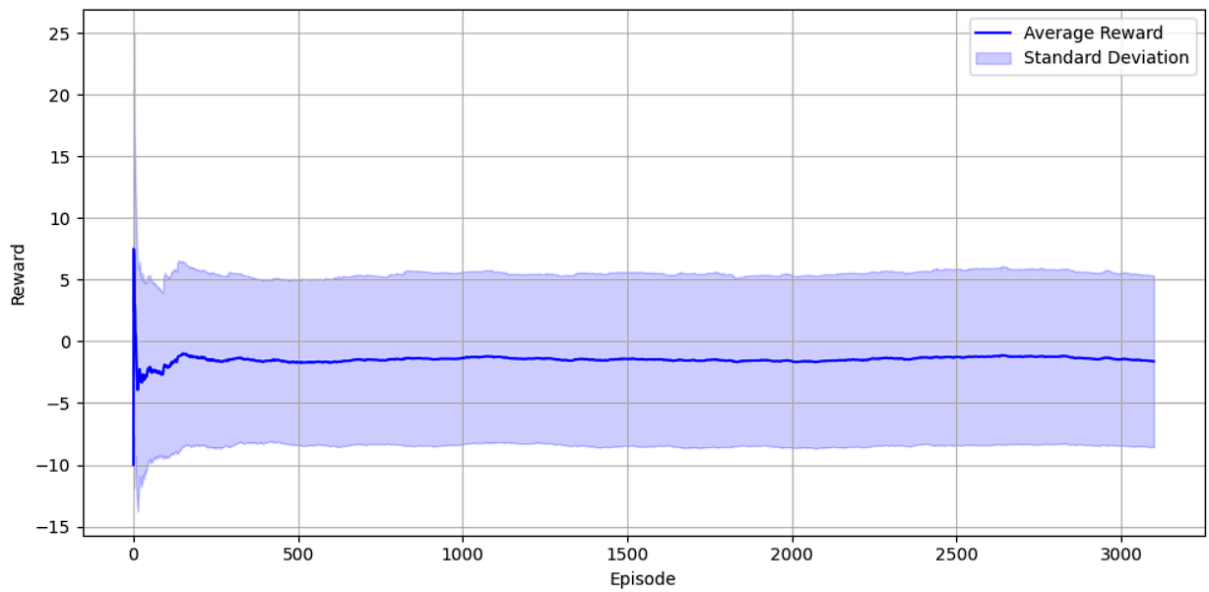


Figure 10: Performance after unfreezing BERT

Changes to the general architecture, including adding more depth to submodels and tuning hyperparameters, showed promising initial results but the problem of high variance remained a persistent challenge.

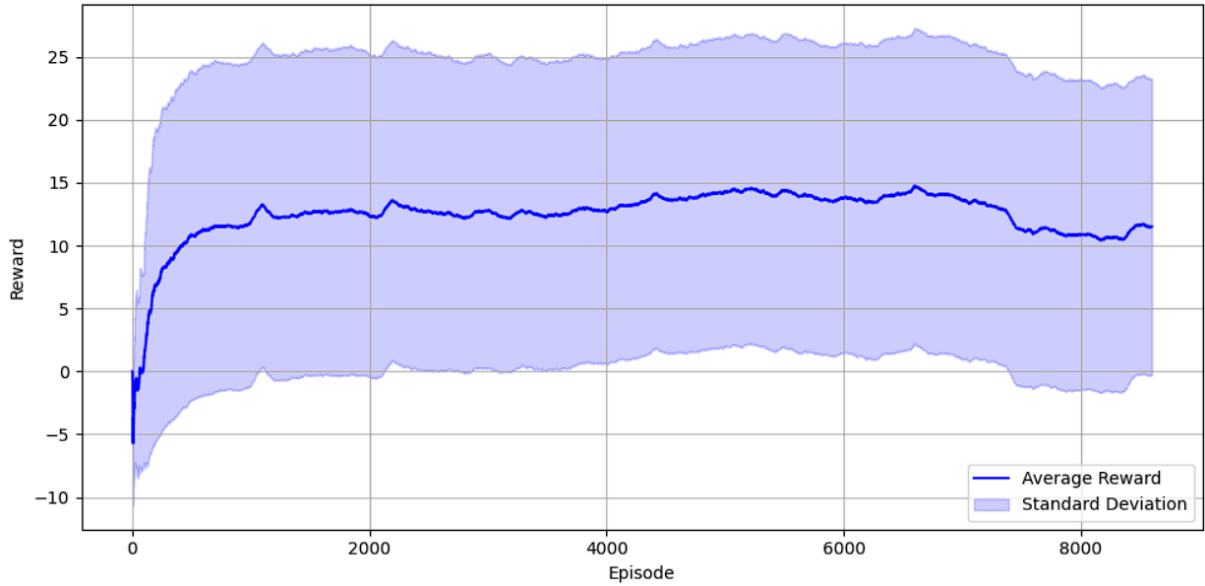


Figure 11: Results with architectural improvements and hyperparameter tuning

The breakthrough came after identifying and solving a key architectural issue: the presence of double stochasticity caused by using both epsilon-greedy exploration and sampling from the probability distribution output of the action decoder. By removing the epsilon-greedy strategy and relying solely on sampling, we observed significant improvement. Following additional hyperparameter tuning, we achieved our final results.

For our best run, the model stabilized after 8000 episodes at a reward of 30. By referencing Figure 12 and the model logs, we can observe the agent’s learned strategy: it goes to the forest, climbs the tree, and picks up the egg for a reward of 5, then returns to the house, opens the window and enters through it, gaining a reward of 10. Subsequently, it proceeds to the living room, reveals the passage to the underground via the cellar and enters for a reward of 25, but then gets eaten by a grue, incurring a penalty of -10.

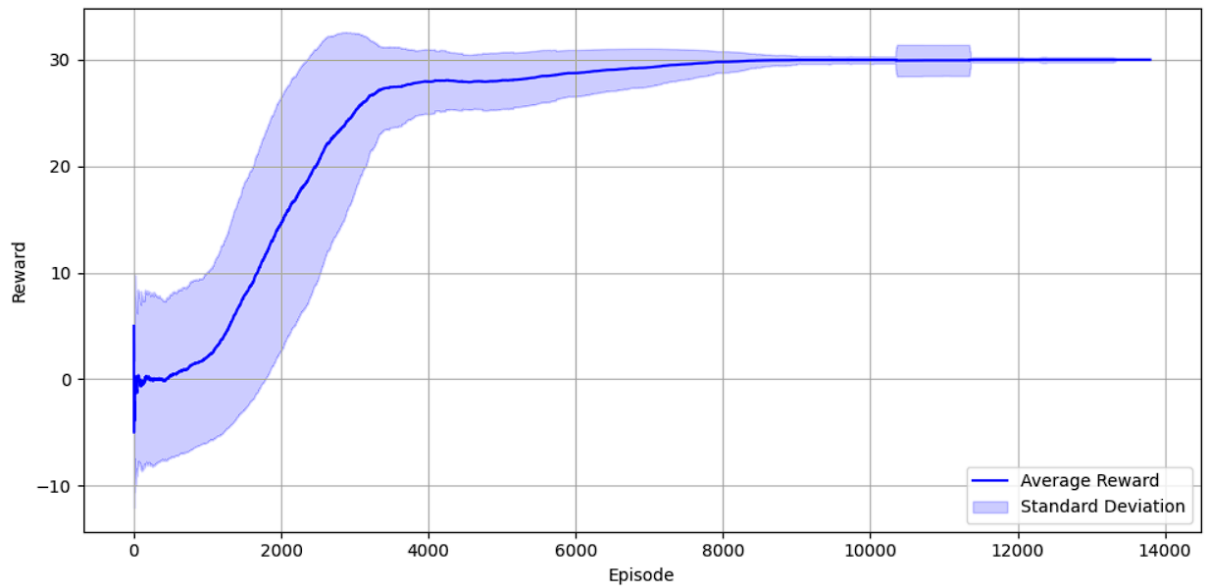


Figure 12: Average Windowed Score during Training in Zork1

This result is particularly interesting because the agent adopts a strategy that combines two

separate routes to maximize reward in the first part of the game. With lower exploration, the agent could have converged to a simpler strategy: either obtaining only +5 by collecting the egg, or directly opening the window to enter the house and continuing on that path for a reward of +25. Instead, the agent intentionally deviates from the main route to collect the smaller reward before returning to the primary path, demonstrating a sophisticated understanding of the environment’s reward structure.

5.4 Ablation Study

The following figure shows an ablation study where we played around with our final architecture:

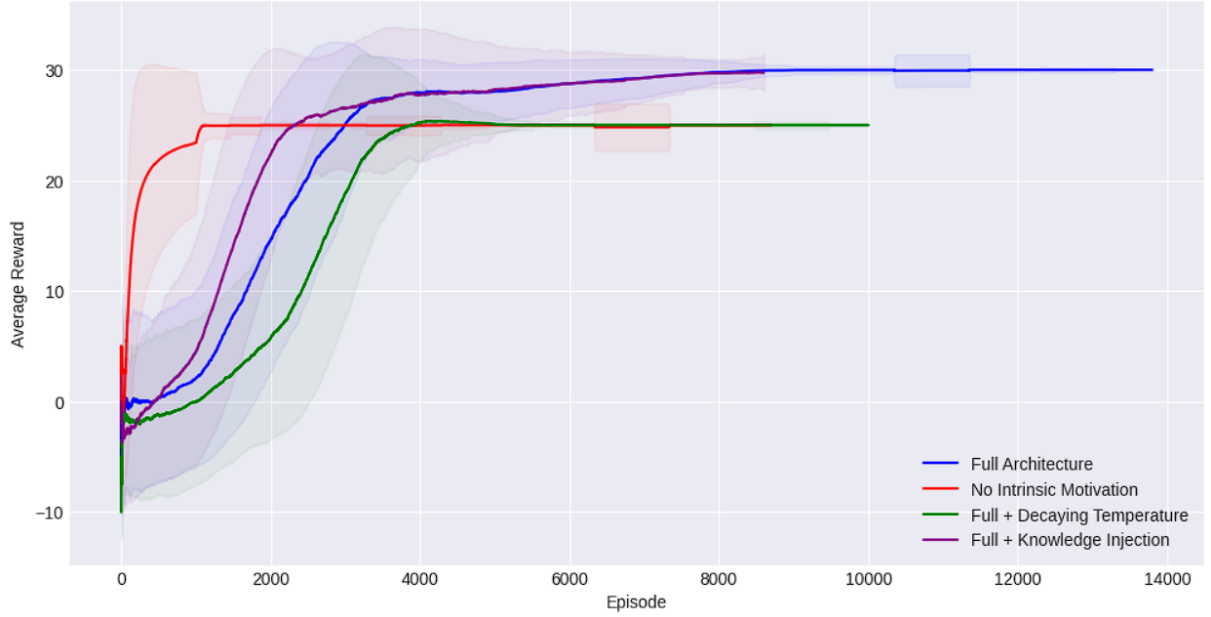


Figure 13: Ablation Study

We can see that the full architecture with and without injecting external knowledge from ConceptNet converged to the same score. Removing intrinsic reward has shown faster convergence but with lower score due to the lack of exploration and the added temperature has given the same results.

6 Conclusion

In this project, we explored how to combine two domains in AI which are Natural Language Processing and Reinforcement Learning in the context of building intelligent agents for playing text-based games which still remains a great challenge and a research subject until nowadays. We built our own architecture by combining relevant techniques from various sources and research papers and implemented it from scratch, we reached near baseline benchmarks and solved problems we faced along the way. Even if the final results were not a breakthrough, we integrated the foundational steps for solving this complex text-based challenge. Future work could emphasize better language understanding, more robust exploration strategies to overcome limited rewards and overall better architecture to capture more complex relationships.

This work highlights the significance of text-based games as a benchmark for developing more general and language-capable AI. By requiring agents to interpret natural language, reason over partially observable states, and plan multi-step actions, these environments simulate real-world cognitive challenges. Our results show that reinforcement learning, when combined with knowledge representation and language understanding, can make meaningful progress in such complex tasks paving the way for more intelligent, adaptable agents in broader applications.

References

- [1] Karthik Narasimhan, Tejas Kulkarni, and Regina Barzilay. Language understanding for text-based games using deep reinforcement learning. In *EMNLP*, 2015.
- [2] He He, Anusha Balakrishnan, Mihail Eric, and Percy Liang. Deep reinforcement learning with a combinatorial action space for predicting popular Reddit threads. In *EMNLP*, 2016.
- [3] Prithviraj Ammanabrolu and Mark Riedl. Playing text-adventure games with graph-based deep reinforcement learning. In *NAACL*, 2019.
- [4] Prithviraj Ammanabrolu and Matthew Hausknecht. Graph constrained reinforcement learning for natural language action spaces. In *ICLR*, 2020.
- [5] Wenlong Huang et al. Language models as zero-shot planners: Extracting actionable knowledge for embodied agents. *arXiv preprint arXiv:2201.07207*, 2022.
- [6] Shunyu Yao et al. ReAct: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022.
- [7] Rahul Chhikara, Tarun Chhabra, Naman Bansal et al. Enhancing text-based game agents with memory of correct actions and commonsense knowledge. In *ACL*, 2023.
- [8] Marc-Alexandre Côté et al. TextWorld: A learning environment for text-based games. In *Workshop on Computer Games, IJCAI*, 2018.
- [9] Matthew Hausknecht, Prithviraj Ammanabrolu, Aniruddha Naik et al. Interactive Fiction Benchmark: Jericho. In *AAAI*, 2020.
- [10] Minjie Wang, Lingfan Yu, Da Zheng et al. Deep Graph Library: Towards efficient and scalable deep learning on graphs. *arXiv preprint arXiv:1909.01315*, 2019.
- [11] Colin Raffel, Noam Shazeer, Adam Roberts et al. Exploring the limits of transfer learning with a unified text-to-text transformer. *JMLR*, 21(140):1–67, 2020.
- [12] Michael Schlichtkrull, Thomas N. Kipf, Peter Bloem et al. Modeling relational data with graph convolutional networks. In *ESWC*, pages 593–607, 2018.
- [13] Yujia Li, Daniel Tarlow, Marc Brockschmidt, and Richard Zemel. Gated graph sequence neural networks. In *ICLR*, 2016.
- [14] Thomas N. Kipf and Max Welling. Variational Graph Auto-Encoders. *arXiv preprint arXiv:1611.07308*, 2016.
- [15] Volodymyr Mnih, Adria Puigdomenech Badia, Mehdi Mirza et al. Asynchronous methods for deep reinforcement learning. In *ICML*, pages 1928–1937, 2016.
- [16] Richard S. Sutton and Andrew G. Barto. Reinforcement Learning: An Introduction. MIT Press, 2018.
- [17] Adam Paszke et al. PyTorch Documentation: torch.nn.Embedding. <https://pytorch.org/docs/stable/generated/torch.nn.Embedding.html>, 2024.